# AI-HAZOP-8800 MVP Architecture

This document captures the complete proposed MVP architecture for extending the existing HazopLLM core into an AI-HAZOP-8800 workflow. It is written as an implementation-oriented design note: what the user provides, what the system loads from the methodology, what each L-phase takes as input, what each L-phase returns, how branching and repair work, and what the final worksheet output contains.

## 1. Core Idea

The existing HazopLLM core should not be discarded. The goal is to keep the current engine and adapt the domain model from classic HAZOP to AI-HAZOP-8800.

```
Keep from existing HazopLLM:
- LangGraph
- Validators
- Reviewers
- Repair loops
- Holistic reviewer
- CSV export
- HTML export
- RAG (optional)
```

```
Old flow:
Function
-> Deviation
-> Cause
-> Effect
```

```
New AI-HAZOP-8800 flow:
AI Component
-> Failure Mode
-> Hazard
-> Risk
-> Safety Goal
-> Requirements / Measures
-> Evidence
```

## 2. User Inputs

The user provides only the analysis context. These are the fields that replace the old single "function" input.

```
User Input:
- component
- component_class
- aspect
- ODD
- scenario
```

### Example

```
component:
Camera Object Detection

component_class:
Camera-ObjectDetection

aspect:
Output

ODD:
Campus Shuttle
Max 15 km/h
Daylight
No dense fog

scenario:
Cyclist behind parked van
```

The practical replacement for the old Function is not only component. The new analysis unit is component + aspect, while ODD and scenario provide the operational context.

# 3. System Inputs / Fixed Catalogues

The following are not entered manually by the user for each run. They are fixed system catalogues extracted from the AI-HAZOP-8800 methodology document.

```
System Inputs (Fixed):
- guidewords
- class_questions
- risk_model
- acceptance_criterion
- mitigation_taxonomy
- evidence_catalogue
```

Recommended internal catalogue files:

```
src/ai_hazop_8800/catalogues/
- taxonomy.yaml
- guidewords.yaml
- class_questions.yaml
- risk_model.yaml
- acceptance_criterion.yaml
- mitigation_taxonomy.yaml
- evidence_catalogue.yaml
```

# 4. Pipeline Overview

```
L1 Failure Mode
-> L2 Hazard
-> L3 Initial Risk
-> L4 Acceptance
-> L5 AI Safety Goals
-> L6 Requirements / Measures
-> L7 Residual Risk
-> L8 Evidence
```

Eight L-phases are justified because each phase depends on the result of the previous one. Combining them would reduce traceability and make repair logic harder.

# 5. L-Phase Inputs and Outputs

## L1 Failure Mode

```
inputs:
- component
- component_class
- aspect
- ODD
- scenario
- guideword
- class_questions

outputs:
- failure_mode
```

L1 applies one fixed guideword at a time to the AI component context. The class_questions focus the model on class-specific risks, but the guideword remains fixed by the system catalogue.

## L2 Hazard

```
inputs:
- component
- component_class
- aspect
- ODD
- scenario
- guideword
- failure_mode
```

```
outputs:
- hazardous_behavior
- potential_harm
- potentially_dangerous
```

L2 converts the component-level failure mode into vehicle-level unsafe behavior and potential harm. It also marks whether the row is safety-relevant enough to continue through the full workflow.

## L3 Initial Risk

```
inputs:
- hazardous_behavior
- potential_harm
- risk_model

outputs:
- E
- PF
- PND
- PNM
- S
- initial_risk
- risk_rationale
```

The LLM should select the risk factors from an allowed scale. The code, not the LLM, calculates the final risk value.

```
R = E x PF x PND x PNM x S
```

## L4 Acceptance

```
inputs:
- initial_risk
- acceptance_criterion

outputs:
- safety_decision
- acceptance_rationale
```

Possible safety decisions:

```
ACCEPT
IMPROVE
RESTRICT
INVESTIGATE
```

## L5 AI Safety Goals

```
inputs:
- failure_mode
- hazardous_behavior
- potential_harm
- safety_decision

outputs:
- ai_safety_goals[]
```

More than one AI Safety Goal is allowed for a single hazard row. For an MVP, it is reasonable to limit the number of goals to avoid excessive output width.

## L6 Requirements / Measures

```
inputs:
- ai_safety_goals[]
- failure_mode
- hazardous_behavior
- mitigation_taxonomy

outputs:
- respecifications[]
- safety_functions[]
- passive_operational_measures[]
```

The measures should be written in requirement style. A separate AI Safety Requirements layer is not required for the MVP because Safety Functions, Respecifications, and Passive / Operational Measures can already be expressed as requirements.

```
Example style:
SF-001: The system shall detect insufficient cyclist observability and output UNKNOWN instead of clear.
R-001: The model shall be validated on occluded cyclist scenarios representative of the target ODD.
P-001: Operation shall be restricted in dense fog or unsupported visibility conditions.
```

### L7 Residual Risk

```
inputs:
- initial_risk
- respecifications[]
- safety_functions[]
- passive_operational_measures[]

outputs:
- residual_risk
- residual_risk_rationale
- residual_status
```

Residual risk is evaluated after the proposed measures. The same risk formula is used, but the factors should reflect the expected effect of the measures.

### L8 Evidence

```
inputs:
- ai_safety_goals[]
- respecifications[]
- safety_functions[]
- passive_operational_measures[]
- evidence_catalogue

outputs:
- evidence[]
- open_assumptions[]
```

Evidence should be linked to the proposed measures. For example, a monitor should be supported by fault injection or runtime verification, while retraining should be supported by dataset audit and scenario validation.

## 6. Conditional Graph Flow

### Safety relevance check after L2

```
L1 Failure Mode
-> L2 Hazard
-> potentially_dangerous?

If NO:
  END

If YES:
  L3 Initial Risk
  -> L4 Acceptance
```

Not every generated failure mode needs to go through all downstream phases. If L2 determines that the item has no safety relevance, the workflow can terminate early.

### Acceptance decision after L4

```
L4 Acceptance
-> safety_decision?

If ACCEPT:
  L8 Evidence
  -> END

If IMPROVE / RESTRICT / INVESTIGATE:
  L5 AI Safety Goals
```

```
  -> L6 Requirements / Measures
  -> L7 Residual Risk
  -> L8 Evidence
  -> END
```

If the initial risk is accepted, the workflow does not need to generate new safety goals, measures, or residual risk. It may still produce evidence and assumptions to document why the risk was accepted.

## 7. Repair and Regeneration Logic

The existing HazopLLM repair strategy should be preserved. The highest priority is the earliest invalid phase, because every downstream phase depends on it.

```
Repair priority:
L1 > L2 > L3 > L4 > L5 > L6 > L7 > L8

If L1 changes:
  invalidate L2-L8
  regenerate downstream

If L2 changes:
  invalidate L3-L8
  regenerate downstream

If L3 changes:
  invalidate L4-L8
  regenerate downstream

... and so on.
```

This preserves traceability. If an earlier safety statement changes, all dependent downstream fields are automatically invalidated and regenerated.

## 8. Holistic Reviewer

The holistic reviewer should remain in the system because it is one of the strongest parts of the existing HazopLLM core.

```
Holistic reviewer checks consistency across:
Failure Mode
-> Hazardous Behavior
-> Initial Risk
-> Acceptance
-> AI Safety Goals
-> Requirements / Measures
-> Residual Risk
-> Evidence
```

It should detect contradictions, broken logic, missing links, and mismatched evidence. Repair requests should be sent to the earliest faulty phase.

## 9. Final Worksheet Output

The final output must match the worksheet-oriented structure. Each final worksheet row is assembled from user inputs, fixed system catalogues, and L1-L8 outputs.

```
Final Worksheet Columns:
- Component
- Class
- Aspect
- ODD
- Scenario
- Guideword
- Failure Mode
- Hazardous Behavior
- Potential Harm
- Initial Risk
- Acceptance Criterion
- Safety Decision
- AI Safety Goal(s)
```

```
- Measures
- Residual Risk
- Evidence
- Open Assumptions
```

### Worksheet Field Mapping

| Worksheet Field | Source |
|---|---|
| Component | User input |
| Class | User input / system taxonomy |
| Aspect | User input |
| ODD | User input |
| Scenario | User input |
| Guideword | System guideword catalogue |
| Failure Mode | L1 |
| Hazardous Behavior | L2 |
| Potential Harm | L2 |
| Initial Risk | L3 |
| Acceptance Criterion | System fixed catalogue / project config |
| Safety Decision | L4 |
| AI Safety Goal(s) | L5 |
| Measures | L6 |
| Residual Risk | L7 |
| Evidence | L8 |
| Open Assumptions | L8 |

## 10. Implementation Notes

Implementation should be done by extending the existing HazopLLM core rather than rewriting it. The main development work is expected in data models, prompt templates, validators, catalogue extraction, graph nodes, and export formatting.

```
Main implementation tasks:
1. Extract methodology catalogues from the PDF into YAML files.
2. Add AI-HAZOP-8800 mode to CLI / GUI.
3. Define AIHazopInput and L1-L8 result models.
4. Add L1-L8 prompt templates.
5. Add validators for each phase.
6. Add conditional graph edges.
7. Reuse repair / review / holistic logic.
8. Generate Worksheet Table output as CSV and HTML.
```

## 11. Development Effort Estimate

```
MVP demo:
2-5 focused days

Practical PoC:
1-2 weeks

Main risks:
- prompt templates
- risk factor scale
```

```
- validation quality
- worksheet export quality
- consistency across L1-L8
```

## 12. Final Architectural Statement

The proposed design keeps the existing HazopLLM engine and extends it into an AI-HAZOP-8800 assistant. The system receives a component-level AI analysis context, applies methodology catalogues from the document, generates a traceable safety analysis through L1-L8, supports conditional execution, applies repair priority from earliest to latest phase, and produces a worksheet-style output suitable for review and demonstration.